

07: Steps Involved in JDBC App (Practical)

The guide outlines the practical steps required to develop a JDBC (Java Database Connectivity) application in Java.

1. Import Required Package and Add JAR File:

- The java.sql package contains essential classes and interfaces for database operations.

```
import java.sql.*;
```

- Download the JDBC driver (e.g., MySQL Connector/J) and add the JAR file to your project's build path. This enables your Java application to communicate with the database.

2. Load and Register the Driver:

The driver converts Java calls into database-specific calls, allowing your application to interact with the database.

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

- It loads the MySQL JDBC driver into memory, which registers it with the DriverManager.

3. Establish the Connection:

Create a connection to the database using the DriverManager.getConnection() method.

- **DriverManager**: This class manages the set of JDBC drivers that can be used to connect to databases. It attempts to establish a connection to the database by calling the appropriate driver.
- **getConnection()**: This method is used to establish the actual connection with the database. It returns a Connection object, which represents an active session with the database.

Parameters:

url: The full address where the database resides

user: The username for the database

password: The password for the database

```
String url = "jdbc:mysql://localhost:3306/jdbclearning";  
String user = "root";  
String password = "your_password";  
Connection connect = DriverManager.getConnection(url, user, password);
```

- In the URL string, **localhost** is used when the database resides on the same system as the application. If the database is hosted on a different system, you can replace localhost with the appropriate **IP address** of that system. The number **3306** represents the port number on which the MySQL database is listening, and **jdbclearning** is the name of the database you are connecting to.

4. Create a Statement Object:

A Statement object is needed to execute SQL queries against the database.

```
Statement statement = connect.createStatement();
```

- **connect.createStatement()** creates a Statement object that is associated with the active database connection. This Statement object allows the execution of SQL queries and commands on the database.

5. Execute the Query:

Use the Statement object to execute SQL queries, such as SELECT, INSERT, UPDATE, or DELETE.

6. Process the Result:

After executing a query, process the results returned by the database.

7. Close the Resources:

Properly closing the database connection and other resources is essential to avoid memory leaks.

```
resultSet.close();
```

```
statement.close();  
connect.close();
```

Source Code:

```
package com.telusko.jdblearning;  
import java.sql.*;  
public class LaunchApp1  
{  
    public static void main(String[] args) throws ClassNotFoundException,  
    SQLException  
    {  
        // Load and Register the Driver  
        Class.forName("com.mysql.cj.jdbc.Driver");  
        // Establish the connection  
        String url = "jdbc:mysql://localhost:3306/jdbclearning";  
        String user = "root";  
        String password = "root123";  
        Connection connect = DriverManager.getConnection(url, user, password);  
        // Creating statement  
        Statement statement = connect.createStatement();  
        // Execute query  
        // Process the result  
        // Close the resources  
        statement.close();  
        connect.close();  
    }  
}
```